POLIMORFISMO

Programación Orientada a Objetos — Cuatro Pilares

DESARROLLO DE SISTEMAS

LOS CUATRO PILARES DE LA POO

ABSTRACCIÓN

Ocultar detalles y mostrar solo lo esencial al usuario.

ENCAPSULAMIENTO

Proteger el estado interno del objeto mediante acceso controlado.

HERENCIA

Reutilizar comportamiento a través de relaciones padre-hijo.

POLIMORFISMO ★

Un mismo mensaje, múltiples comportamientos según el objeto receptor.

¿QUÉ ES EL POLIMORFISMO?

Definición

Del griego: poly (muchos) + morphos (formas). En POO, es la capacidad de un objeto de tomar distintas formas.

Un mismo método puede comportarse de manera diferente según el tipo real del objeto que lo ejecuta en tiempo de ejecución.

Ejemplo: lista.add() funciona igual si lista es un ArrayList o un LinkedList. El comportamiento cambia; la interfaz no.

Una relación esencial

El polimorfismo y la herencia están profundamente relacionados. La herencia crea jerarquías de clases; el polimorfismo las aprovecha.

Con herencia, una variable de tipo Figura puede referenciar un Circulo, un Rectangulo o un Triangulo. El método calcularArea() se ejecuta según el tipo real del objeto.

Regla: si B extiende A, entonces B es un A. Esto habilita el polimorfismo: podés tratar un B como si fuera un A, y el método correcto se invoca automáticamente.

TIPOS DE POLIMORFISMO EN JAVA

TIEMPO DE COMPILACIÓN

- **Sobrecarga de Métodos**

También llamado Overloading. Múltiples métodos con el mismo nombre pero diferente firma.

- **Se resuelve en compilación**

El compilador decide qué versión del método invocar según los parámetros.

- **Ejemplo en Java**

sumar(int,int) vs sumar(double,double) vs sumar(int,int,int)

TIEMPO DE EJECUCIÓN

- **Sobreescritura de Métodos**

También llamado Overriding. Una subclase redefine un método heredado de su clase padre.

- **Se resuelve en ejecución**

La JVM decide en tiempo de ejecución qué implementación usar según el tipo real del objeto.

- **Ejemplo en Java**

Figura.calcularArea() redefinido en Circulo, Rectangulo y Triangulo.

SOBRECARGA DE MÉTODOS (OVERLOADING)

Polimorfismo en Tiempo de Compilación

Mismo nombre de método, diferente firma. El compilador elige cuál versión invocar según el tipo y cantidad de parámetros.

```
public class Calculadora {
    int sumar(int a, int b)          { return a + b; }
    double sumar(double a, double b) { return a + b; }
    int sumar(int a, int b, int c)   { return a + b + c; }
}

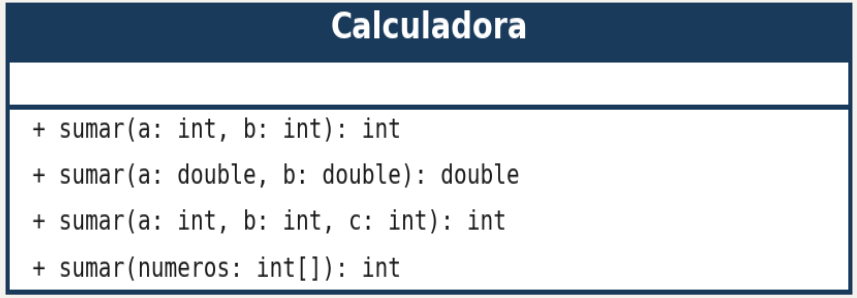
c.sumar(1, 2);       // invoca sumar(int, int)
c.sumar(1.5, 2.5);   // invoca sumar(double, double)
```

SOBRECARGA (OVERLOADING) — DIAGRAMA UML

Clase Calculadora con métodos sobrecargados

La misma clase Calculadora expone cuatro firmas distintas para sumar().

La firma incluye: nombre del método + tipos y orden de los parámetros (NO el tipo de retorno).



Mismo nombre, diferente firma → el compilador elige cuál invocar

SOBREESCRITURA DE MÉTODOS (OVERRIDING)

Polimorfismo en Tiempo de Ejecución

Una subclase redefine un método heredado de su clase padre usando @Override. La anotación es opcional pero recomendada: el compilador verifica que realmente se está sobreescribiendo.

```
abstract class Figura {
    abstract double calcularArea();
}

class Circulo extends Figura {
    @Override
    public double calcularArea() {
    }
}
```

POLIMORFISMO EN ACCIÓN — CÓDIGO JAVA



Variable de tipo padre → objeto de tipo hijo

```
Figura f1 = new Circulo(5);      // ref. Figura, obj. Circulo

Figura f2 = new Rectangulo(4,6); // ref. Figura, obj. Rectangulo

f1.calcularArea(); // → 78.54 (PI * 5²) — usa Circulo.calcularArea()
f2.calcularArea(); // → 24.0 (4 × 6)  — usa Rectangulo.calcularArea()

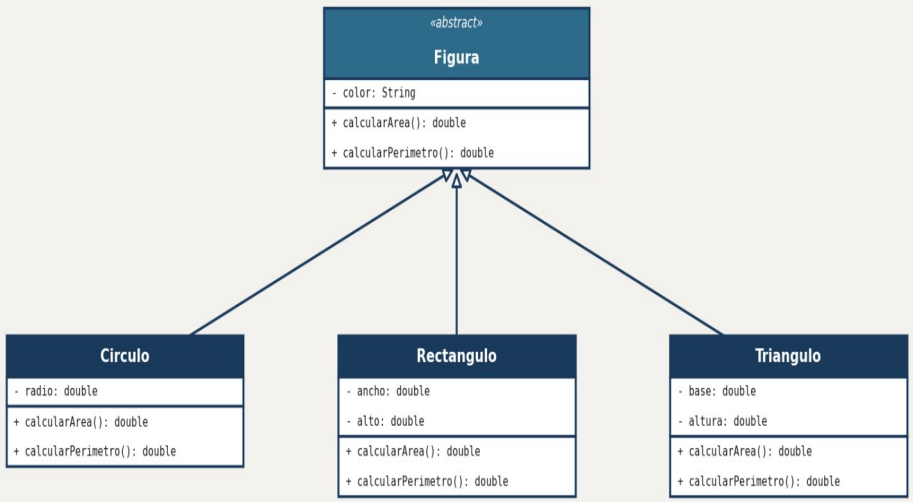
// La referencia es Figura, pero Java llama al método
// del OBJETO REAL en tiempo de ejecución (dynamic dispatch)
```

OVERRIDE — DIAGRAMA DE CLASES UML

Clase abstracta Figura y sus subclases

Figura es abstracta: no se puede instanciar directamente.

Cada subclase provee su propia implementación de calcularArea() y calcularPerimetro().



La interfaz como contrato de comportamiento

En Java, una interfaz define un contrato: qué métodos debe implementar una clase, sin decir cómo. Esto habilita polimorfismo sin herencia de clase.

```
// Con ArrayList:
List<String> lista = new ArrayList<>();
lista.add("Hola");

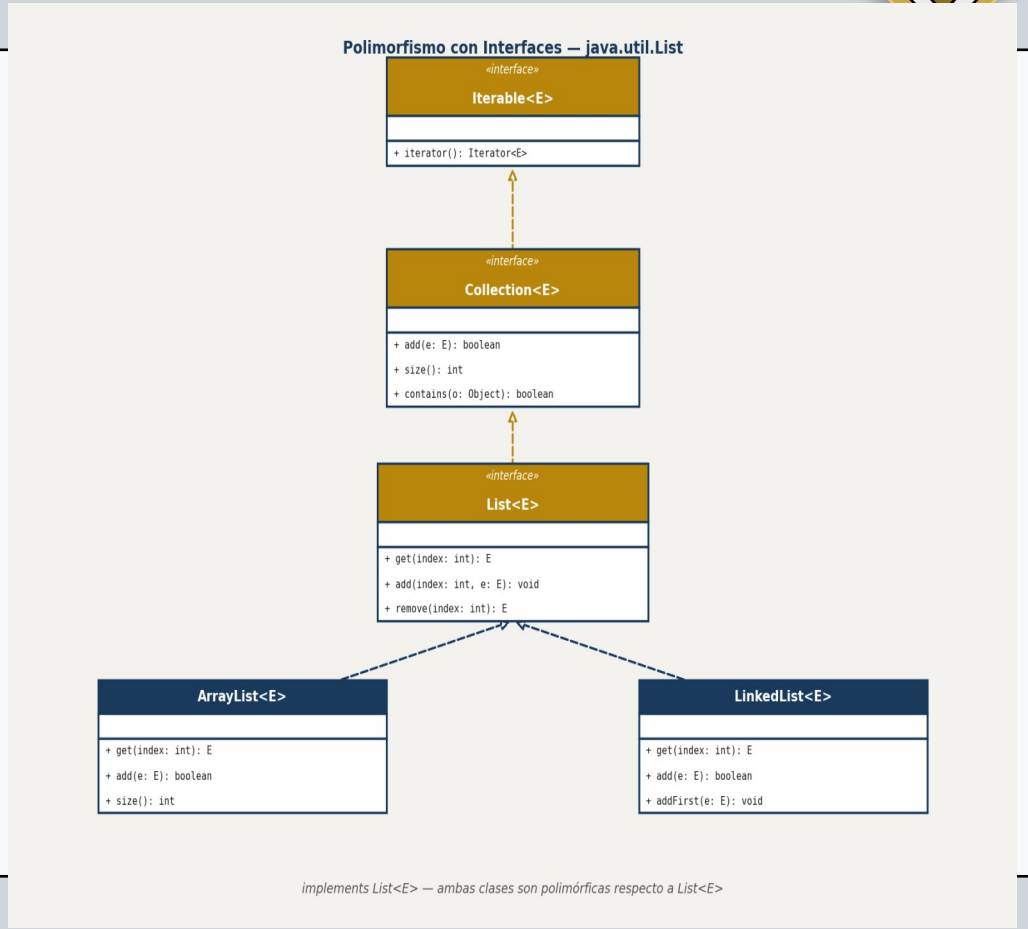
// Cambiamos implementación:
List<String> lista = new LinkedList<>();
// El resto del código NO cambia.
```

INTERFACES — DIAGRAMA DE CLASES UML

java.util.List y sus implementaciones

List<E> extiende Collection<E> que extiende Iterable<E>.

ArrayList y LinkedList implementan List<E> con distintas estructuras internas.



USO POLIMÓRFICO DE List EN JAVA

CÓDIGO JAVA

- **Declaración polimórfica**

```
List<String> nombres = new ArrayList<>();
```

- **Operaciones comunes**

```
nombres.add("Ana"); nombres.get(0); nombres.size();
```

- **Cambiar implementación**

```
List<String> nombres = new LinkedList<>(); // misma interfaz
```

¿QUÉ PASA?

- **Tipo de referencia**

La variable es de tipo List (interfaz). No importa si es ArrayList o LinkedList.

- **Comportamiento polimórfico**

add() y get() se ejecutan en la implementación real, pero el código cliente no cambia.

- **Ventaja clave**

Si mañana cambiamos a LinkedList, el resto del código no necesita modificarse.

¿POR QUÉ USAR POLIMORFISMO?

CÓDIGO REUTILIZABLE

Métodos genéricos que funcionan con cualquier subtipo. Un método que acepta Figura funciona con Circulo, Rectangulo y cualquier figura futura.

EXTENSIBILIDAD

Agregar nuevos tipos sin modificar código existente. Nueva clase Hexagono extiende Figura y automáticamente es compatible con todo el sistema.

BAJO ACOPLAMIENTO

El código depende de abstracciones (interfaces/clases abstractas), no de implementaciones concretas. Más fácil de testear y mantener.

PRINCIPIO ABIERTO/CERRADO

Abierto para extensión, cerrado para modificación. Se pueden agregar nuevos comportamientos extendiendo clases sin tocar el código existente.



RESUMEN

“Un programa polimórfico está preparado para el futuro.” Overloading: mismo nombre, diferente firma → resuelto en compilación. Overriding: redefinir método heredado → resuelto en ejecución. Interfaces: polimorfismo sin herencia de clase.